

Changes to dashboard and db

📅 Date July 31, 2026 → August 3, 2026

Changes made to the dashboard:

1. Broadened the CSS selector that disables the flicker/fade effect

Before:

CSS

```
div[data-testid="stFragment"] {  
  opacity: 1 !important;  
  transition: none !important;  
}
```

Now:

CSS

```
div[data-testid="stFragment"],  
div[data-testid="stDataFrame"],  
div[data-testid="stVerticalBlock"],  
div[data-testid="stElementContainer"] {  
  opacity: 1 !important;  
  transition: none !important;  
}
```

Streamlit applies its dimming effect to individual widgets during a rerun, not just the outer fragment wrapper. The old rule only stopped the outer container from fading — the actual dataframe grid and content blocks inside it were still fading independently. This adds those specific element types to the same "never fade" rule.

2. Slowed the auto-refresh interval

Before:

python

```
@st.fragment(run_every=20)
```

Now:

python

```
@st.fragment(run_every=30)
```

Since your actual data ingestion is throttled to once every 15 seconds regardless (via `_maybe_ingest()`), refreshing the whole page every 20 seconds was doing extra re-renders that couldn't even show new data half the time. Bumping it to 30 seconds means fewer total re-renders per minute, and therefore fewer chances for any flicker to show up at all — without losing any real freshness, since new data was never available more often than the 15-second ingestion throttle allows anyway.

Everything else — the alert table, filters, review buttons, charts, enrichment display, all the logic — is completely unchanged. Go ahead and test it; let me know if the flicker is better, and we'll decide whether the fingerprint-tracking or hide-the-status-widget changes from before are worth adding on top.

What happens when new alerts actually arrive

Every 30 seconds, the fragment reruns regardless of whether anything changed (that's what `run_every=30` means — a fixed timer, not "only when there's new data"). When it reruns:

1. `_maybe_ingest()` checks if 15 seconds have passed since the last real ingestion — if yes, it reads any new lines from `eve.json` and writes them into `alerts.db`.
2. `load_alerts_from_db()` re-queries the database — since this is `@st.cache_data(ttl=5)`, if 5+ seconds have passed since the last query, it runs a fresh SQL query and gets back whatever's currently in the table, including any brand-new rows.
3. The dataframe, KPI numbers, and charts all rebuild using this fresh data.

With the opacity fix applied: the actual pixels update — new rows appear in the table, KPI counts change, chart bars adjust — but there's **no gray/dim overlay flashing over the content while this happens**. The old default Streamlit behavior would visibly gray out the whole section for a moment during the rerun, even for a tiny, fast update; now that's suppressed, so the transition from "5 alerts" to "6 alerts" should look like the numbers/rows just directly change, ideally without an intermediate faded frame.

The two feature additions

- 1. Store the matched enrichment JSON directly in the alerts table** — so each alert row carries its own snapshot of whatever rule-catalog data matched it at ingestion time, rather than always doing a live lookup.
- 2. Hide null/empty fields in the dashboard** — currently the "Rule Catalog Information" section always prints every field with an `'N/A'` fallback; I'll make it only show fields that actually have a value.